PROJECT AND TEAM INFORMATION

Project Title

IntelliPlag: A DSA and OOP's based Plagiarism Detection System Using Trie, Hashing and String Matching

Student / Team Information

Team Name: *Team ID:*	*IntelliPlag* *DSCPP-III-2026-T069*
Team member 1	*Sinha, Shivya – 2510380257* *Email: 2510380257@geu.ac.in* 
Team member 2	*Kuletha, Srishti – 2510012545* *Email: 2510012545@geu.ac.in* 
Team member 3	*Jagwan, Antriksh Singh – 2512510006* *Email: 2512510006@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

The increasing availability of digital documents, assignments, articles and source code has made the detection of copied content a very important requirement in academic and professional environments. Manually comparing documents to identify similarities is quite time-consuming, especially when dealing with large files or multiple documents. Simple comparison methods may also overlook repeated phrases that occur at different locations within the documents.

IntelliPlag is a DSA and OOPs based solution for efficiently identifying similarities between text documents. The system processes the input documents, performs preprocessing and tokenization and generates sequences of words using n-grams then compares these sequences to identify potentially copied content. Data structures such as Tries, hash tables, vectors, sets, and linked lists are used to organize and search the processed information. String matching and slidingwindow techniques further support the identification of common phrases.

The project is motivated not only by problem of plagiarism detection but also by the need to show how fundamental DSA concepts can be applied to real-world problems. OOP provides a modular architecture in which document processing, searching, similarity calculation and report generation can be done as separate and reusable components. The goal of this system is to provide a clear similarity score and a report of detected matches making the detection process faster, organized and easy to understand.

State of the Art / Current Solution

Many standard plagiarism detectors rely heavily on web-scale crawling, distributed vector search engines and neural semantic embeddings like Transformer-based models. While these deeplearning architectures excel at getting highlevel semantic paraphrasing they introduce significant computational, non deterministic outputs and high build costs. As a result exact sequence matching and comparison rely on foundational (DSA) for low-latency, deterministic and resource-efficient processing. Modern algorithmic approache balances search efficiency and index size using rolling hash methods, structural string matching and space optimized index structures. Algorithms like RabinKarp leverage rolling hash functions to evaluate candidate substrings in linear time. While structural algorithms like Knuth Morris Patt (KMP) eliminate repetitive character comparisons through pre assigned prefix tables. Complementing these algorithms are prefix trees (Tries) which provide linear time retrieval for continuous character sequences of length. The Winnowing algorithm selects subset fingerprints from overlapping n grams to minimize storage requirements without compromising match sensitivity.

Project Goals and Milestones

The primary goal of IntelliPlag is to build a simple, reliable and fast plagiarism detection program. Instead of using complex artificial intelligence or pre made web search engines this project uses core DSA and OOP concepts. By building every part of the system manually i.e, including data storage, searching methods and score calculations the project shows how basic computer science principles can solve realworld problems.

Ingestion and Text preprocessin. The first milestone focuses on reading files and cleaning the incoming text. The program takes raw document files turns all text into lowercase letters and removes unnecessary elements like punctuation, symbols, etc.

Building core data structures. The second milestone involves building custom data structures. These include a dynamic Vector to hold words in order a Hash Table to quickly store and look up unique word patterns and a trie to search for matching sentence prefixes. Building these custom modules gives full control over system memory and speed.

Sequence chunking and indexing. The third milestone focuses on splitting document text into small phrase peices called n.grams. The program creates unique numerical fingerprints for these word combinations and saves them inside the custom Hash Table and trie so that fast comparisons can be made later.

String matching and similarity calculation. The fourth milestone involves comparing documents to find same sentences. The algorithm checks how many n.gram fingerprints are matching between files and calculates a similarity percentage using mathematical formulas like jaccard similarity.

Ranking engine and report generation. The final milestone takes the results and presents them clearly to the user. Matches are sorted using a MaxHeap priority queue so that the most heavily plagiarized documents appear at the top.

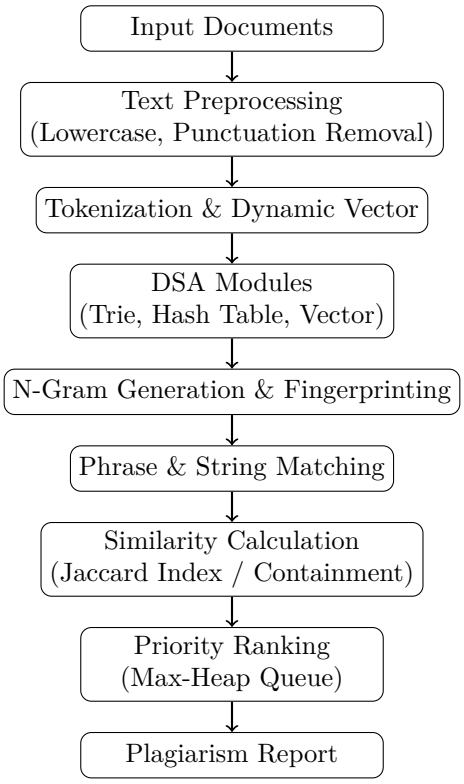
Project Approach

The intelliPlag system is designed using OOP principles where each stage of the plagiarism detection process is handled by a dedicated class. Instead of putting all the code into a single file the system breaks the workflow into seperate components. This keeps the program organized easy to debug and simple to maintain. The overall execution flows smoothly from reading raw files to generating a final report with each class passing its processed data directly to the next step in the pipeline.First the system uses a Document class to load and hold raw text from input files. This raw text is immediately passed to a Preprocessor class which cleans up the content by removing punctuation stripping out extra spaces,and converting every letter to lowercase. Once the text is clean the Preprocessor converts the words into a continuous list of tokens and passes them to the NGramGenerator class. The NGramGenerator gives a fixed window across the list of words to create short overlapping phrases called n.grams. Breaking the text into these small chunks allows the system to detect copied phrases even if an author changes surrounding words or shifts sentences around.Next the generated n.grams are sent to an IndexEngine class which acts as the manager for all custom DSA.

The system evaluates and ranks the results using dedicated classes. A SimilarityEngine class prompts the custom hash table and trie to count how many phrase overlap between documents. It then calculates an overall matchings score using formulas like jaccard similarity. These similarity scores are passed to a Ranker class which uses a custom MaxHeap priority queue to automatically sort the results so that the highest similarity matches stay at the top. Lastly a ReportGenerator class reads the sorted data from the MaxHeap and gives a clear summary showing match percentages highlighted copied phrases and source file locations.

System Architecture (High Level Diagram)

IntelliPlag System Architecture



The architecture illustrates the IntelliPlag pipeline from document ingestion and preprocessing through core DSA indexing, n-gram matching, similarity calculation, and max-heap priority ranking to generate the final plagiarism report.

Project Outcome / Deliverables

Fully modular plagiarism core. A standalone C++ engine built using OOP sdesign principles free from heavy third-party search dependencies. Algorithmic Library. Memory optimized custom use of a Prefix trie, an n.gram Hash table and a dynamic memory Vector. Offset-Aware Matching Engine. A sequence comparison system capable of locating exact and nearexact phrase matches alongside their structural file offset locations. Formatted Plagiarism Analysis. A detailed report listing pairwise document similarity percentage categorized matching sections and a prioritized severity list of suspicious document pairs.

Assumptions

The system operates under several assumptions to ensure good performance. First all input files are assumed to be plain text formatted in UTF8. Nontext file formats like PDF or docx must be converted prior. Second the system focuses strictly on exact sequence copying and local word rearrangements. it does not detect deep AIgenerated paraphrasing or swapping without shared vocabulary. Third the sliding window size n.gram length is constant during execution typically between 3 and 7 words to effectively balance detection acuracy with false positive suppression. Finally all document processing dynamic vectors hash tables and trie structures are assumed to fit within available system RAM for fast in memory execution.

References

[1] International Journal of Computer Theory and Engineering (IJCTE)

[2] International Journal of Computer Applications (0975 – 8887) Volume 185

[3] NCRTCS - 2026 (Volume 14 – Issue 02) AI-Based Plagiarism Detection System

[4] ACM SIGCSE BulletinVol. 13, No. 1A plagiarism detection system